ADR-CAMP-02 — Algoritmo de sorteio de chaves eliminatórias

Autor(a): a2576295 — Nicolly Kauana Ferreira
Projeto individual: Sistema de Gestão de Campeonatos de Jiu-Jitsu
Stack atual: Node.js 18+ · Express 5 · EJS (SSR) · PostgreSQL 14+ (driver `pg`, SQL parametrizado) · arquitetura em camadas (controllers → services → repositories).

Status

Proposed

Contexto

Depois de fechadas as inscrições, cada categoria precisa de uma **chave eliminatória** (bracket). O sorteio é o momento mais sensível do campeonato: atletas, professores e academias acompanham e contestam qualquer suspeita de favorecimento.

Problema: como gerar a chave de forma que ela seja:

- **justa e imprevisível** (ninguém consegue prever ou manipular os confrontos);
- **conforme as regras do esporte:** chave em potência de 2, cabeças de chave (seeds) em posições oficiais (Seed 1 e Seed 2 só se cruzam na final), BYEs para os seeds e **nunca BYE x BYE**;
- **socialmente aceitável:** evitar que companheiros de equipe se enfrentem logo na 1ª rodada;
- **rápida e reproduzível em teste**, com até 128 atletas.

Por que a decisão é necessária: a estrutura gerada (lutas encadeadas com `proxima_luta`/`proximo_slot`) é a base de todo o restante: avanço automático, rollback de correções, pódio e pontuação.

Restrições:

- tamanho máximo de 128 competidores por chave (RF32);
- geração completa em menos de 100 ms (RNF13);
- o sorteio só pode ser refeito enquanto a chave estiver `NAO_INICIADA` (RF36);
- não pode depender de serviços externos (o evento pode ter internet instável).

Requisitos relacionados

Tipo	Requisitos
Funcionais	RF25 (seeds), RF31 (mínimo 2 confirmados), **RF32 (potência de 2)**, **RF33 (posição fixa dos seeds)**, **RF34 (distribuição de BYEs)**, **RF35 (embaralhamento criptográfico e separação de equipes)**, RF36 (re-sorteio), RF37 (avanço de BYEs), RF39 (árvore visual)
Não funcionais	**RNF13 (até 128 atletas em < 100 ms)**, RNF09 (persistência atômica), RNF21 (testes), RNF24 (conformidade com regras do Jiu-Jitsu)

Tipo	Requisitos
Arquiteturais	Testabilidade (lógica pura), segurança (imprevisibilidade), extensibilidade para novos formatos

Alternativas consideradas

A1 — Sorteio no banco (`ORDER BY random()`)

Os confirmados são buscados já embaralhados pelo PostgreSQL e preenchem os slots em ordem.

- **Vantagens:** uma linha de SQL; nenhum código de algoritmo.
- **Desvantagens:** não respeita seeds, BYEs nem separação de equipes; `random()` do PostgreSQL **não é criptograficamente seguro**.
- **Riscos:** confrontos BYE x BYE; seeds 1 e 2 se cruzando cedo; colegas de equipe se eliminando na 1ª rodada.
- **Impactos:** regras do esporte teriam de ser corrigidas manualmente depois.
- **Limitações:** lógica difícil de testar (depende do banco).

A2 — Embaralhamento simples na aplicação (`Math.random` + `sort`)

Embaralha a lista em JavaScript e distribui sequencialmente.

- **Vantagens:** simples; roda em memória; fácil de entender.
- **Desvantagens:** `sort(() => Math.random() - 0.5)` gera **distribuição enviesada**; `Math.random` é previsível (PRNG não criptográfico); continua ignorando seeds, BYEs e equipes.
- **Riscos:** questionamento de justiça; sorteio potencialmente reproduzível por quem conhece o estado do gerador.
- **Impactos:** mesmo problema de A1 nas regras.
- **Limitações:** não atende RF33–RF35.

A3 — Sorteio manual pelo organizador

O operador posiciona cada atleta na chave pela interface.

- **Vantagens:** controle total; trata casos especiais.
- **Desvantagens:** lento (dezenas de categorias por evento); sujeito a erro humano.
- **Riscos: acusação de favorecimento**, que é justamente o que o sistema quer evitar.
- **Impactos:** interface complexa de arrastar e soltar; aumenta fila no dia do evento.
- **Limitações:** não escala; não é auditável quanto à imparcialidade.

A4 — Algoritmo em memória em etapas: regras determinísticas + aleatoriedade criptográfica + busca do melhor sorteio ☑

Função pura (`chaveGeneratorService.gerarChave`) que:

1. calcula o tamanho (próxima potência de 2, até 128);
2. posiciona os **seeds** em slots fixos pela ordem internacional (`gerarOrdemSeeds`: 8 → `[1,8,4,5,2,7,3,6]`);

3. distribui os **BYEs** priorizando os adversários dos seeds 1, 2, 3... e **valida que nunca há BYE x BYE**;
4. sorteia os demais com **Fisher-Yates usando** `crypto.randomInt`, repetindo até 500 vezes e mantendo a distribuição com **menos confrontos da mesma equipe** na 1ª rodada (para no primeiro sorteio com 0 conflitos);
5. monta a árvore completa de lutas com `proximaRodada`, `proximaPosicao` e `proximoSlot`.

A persistência fica em `chaveService.gerarChave`, numa transação com `SELECT … FOR UPDATE` na categoria.

- **Vantagens:** atende todas as regras; imprevisível (CSPRNG); distribuição uniforme (Fisher-Yates); função pura, testável sem banco; rápida.
- **Desvantagens:** heurística (melhor de N tentativas), sem garantia matemática de ótimo; separa equipes só na 1ª rodada.
- **Riscos:** em categorias em que uma equipe tem mais da metade dos atletas, o conflito é inevitável. O algoritmo devolve o mínimo encontrado e informa `conflitosEquipe`.
- **Impactos:** o gerador vira um componente isolado, reutilizável por outros formatos de chave.
- **Limitações:** só eliminatória simples; 128 atletas no máximo.

A5 — Solver de restrições / backtracking ótimo

Modela o sorteio como problema de satisfação de restrições (CSP) e procura a solução ótima, inclusive separando equipes nas rodadas seguintes.

- **Vantagens:** garante o mínimo de conflitos; pode espalhar colegas pelas metades da chave.
- **Desvantagens:** complexidade de implementação e manutenção alta; tempo de execução exponencial no pior caso; aleatoriedade fica mais difícil de garantir (o solver tende a achar sempre a "mesma" solução).
- **Riscos:** estourar os 100 ms do RNF13; dependência de biblioteca externa.
- **Impactos:** mais difícil de explicar ao público, o que prejudica a percepção de justiça.
- **Limitações:** ganho marginal sobre A4 para chaves de até 128 atletas.

Critérios de decisão

Critério	Peso	A1 SQL random	A2 Math.random	A3 Manual	A4 Em memória + CSPRNG	A5 Solver
Conformidade com regras (seeds, BYE)	5	1	1	4	**5**	5
Justiça / imprevisibilidade	5	2	2	1	**5**	3
Separação de equipes	4	1	1	3	**4**	5
Desempenho (< 100 ms, 128 atletas)	3	5	5	1	**5**	2
Testabilidade	3	1	3	1	**5**	4
Simplicidade de manutenção	3	5	5	3	**4**	1

Critério	Peso	A1 SQL random	A2 Math.random	A3 Manual	A4 Em memória + CSPRNG	A5 Solver
Total ponderado		52	58	52	**108**	81

Decisão

O sorteio de chaves será feito pelo **algoritmo em memória em etapas (A4)**, implementado como **função pura** em `src/services/chaveGeneratorService.js`, sem acesso ao banco. A gravação fica a cargo de `chaveService`, dentro de uma transação. Regras verificáveis:

1. O tamanho é a menor potência de 2 maior ou igual ao número de confirmados (mínimo 2, máximo 128).
2. Seeds devem ser inteiros 1..k sem lacunas nem repetições. Caso contrário, `ValidationError`.
3. Seeds ocupam os slots definidos por `gerarOrdemSeeds`.
4. BYEs vão primeiro para os adversários dos seeds em ordem crescente. Um confronto BYE x BYE gera erro interno.
5. Os demais atletas são embaralhados com **Fisher-Yates + `crypto.randomInt`**, com até 500 tentativas, escolhendo a de menor `conflitosEquipe`.
6. O re-sorteio e a exclusão só são permitidos com a chave `NAO_INICIADA`.

Verificação: `npm run test:chaves` e `npm run test:bracket` cobrem tamanho, ordem de seeds, ausência de BYE x BYE e encadeamento das lutas; `npm run simular` gera campeonatos completos.

Justificativa

- É a **única alternativa que atende ao mesmo tempo** RF32, RF33, RF34 e RF35. As demais falham em regras do esporte (A1, A2), em imparcialidade (A3) ou em desempenho e simplicidade (A5).
- **Justiça demonstrável:** Fisher-Yates gera permutações uniformes, e `crypto.randomInt` usa o gerador criptográfico do sistema operacional, impossível de prever. É um argumento técnico objetivo diante de contestações.
- **Separar o que é regra do que é sorte:** seeds e BYEs são determinísticos (seguem o regulamento), e só a posição dos demais é aleatória. Isso deixa o resultado explicável ao público.
- **Desempenho:** no pior caso, 500 tentativas × 128 slots dá cerca de 64 mil operações simples, executadas em poucos milissegundos, bem abaixo dos 100 ms do RNF13.
- **Testabilidade e evolução:** por ser função pura (entrada: inscritos; saída: slots e lutas), pode ser testada sem banco e trocada ou estendida (por exemplo com o padrão Strategy) para novos formatos como dupla eliminação ou round-robin, sem alterar o service de persistência.

Consequências

Positivas

- Chaves sempre válidas segundo o regulamento (RNF24).
- Sorteio imprevisível e defensável, o que reduz disputas e acusações de favorecimento.
- Menos confrontos entre colegas de equipe na estreia, melhorando a experiência das academias.
- Gerador isolado e testável. A árvore com `proximo_slot` viabiliza o avanço automático (RF37, RF46) e o rollback ordenado (RF49).

- O re-sorteio é barato, já que basta chamar a função de novo.

Negativas

- Heurística sem garantia de ótimo: em casos extremos pode restar conflito evitável (o valor `conflitosEquipe` é retornado para o operador avaliar).
- Separação de equipes **só na 1ª rodada**: colegas ainda podem se cruzar na 2ª rodada. Espalhar por metades da chave exigiria evoluir o algoritmo.
- Por usar CSPRNG, o sorteio **não é reproduzível** por semente, o que impede "refazer o mesmo sorteio" para auditoria. A rastreabilidade depende de registrar o resultado em `logs_auditoria`.
- Limitado a eliminatória simples com até 128 atletas. Outros formatos exigem novos geradores.

Impactos sobre outras equipes

> Por ser um **projeto individual**, não há outras equipes. Esta seção lista os **módulos do próprio sistema** afetados pela decisão.

Módulo / área afetada	Impacto
Inscrições (Módulo 7)	Só inscrições `CONFIRMADA` entram no sorteio, e o seed informado precisa ser válido (RF25, RF31).
Gestão de chaves (Módulo 8, `chaveService`)	Faz a persistência em transação com `FOR UPDATE`, controla o re-sorteio e a exclusão (só com a chave `NAO_INICIADA`) e chama o gerador puro.
Árvore visual (Módulo 9)	Consome a estrutura `rodada/posicao/proximo_slot`. Precisa exibir os BYEs e, se houver, o aviso de conflitos de equipe (RF39–RF41).
Resultados e avanço (Módulo 10)	O encadeamento `proxima_luta/proximo_slot` gerado aqui é o que permite o avanço automático de vencedores e de BYEs (RF37, RF46).
Correção / rollback (Módulo 11)	A árvore encadeada permite verificar se o vencedor já lutou a rodada seguinte (RF49).
Auditoria (Módulo 15)	Como o sorteio não é reproduzível, a geração e o re-sorteio precisam ser registrados no log (RF64).
Testes	Testes unitários sem banco para seeds, BYEs e conflitos, e testes de integração para re-sorteio e bloqueio após o início.